

Structs in C++ — Competitive Programming Notes

Topics covered: Constructors, Methods, Operator Overloading, Sorting with Structs

1. Basic Declaration

A struct groups related variables together under one name.

```
struct Point {  
    int x, y; // member variables  
};
```

```
Point p;          // x and y are garbage – uninitialized  
Point q = {};     // x=0, y=0 – zero initialized (safe)
```

Rule: if no constructor is defined, declaring a variable without {} leaves members uninitialized. Use = {} to zero-initialize safely.

2. Constructors

A constructor is a special function that runs automatically when you create a struct variable. It has the same name as the struct and no return type.

```
struct Point {  
    int x, y;  
  
    Point() : x(0), y(0) {}           // default constructor  
    Point(int x, int y) : x(x), y(y) {} // parameterized constructor  
};
```

```
Point a;          // calls default → x=0, y=0  
Point b(3, 4);    // calls parameterized → x=3, y=4
```

Initializer List Syntax:

```
Point(int x, int y) : x(x), y(y) {}  
//           ^^^^^  ^^^^^  ^^^^^  
//           parameters member = parameter  
  
// Left side of x(x) → member variable  
// Right side of x(x) → parameter  
// Equivalent but clearer with different names:  
Point(int px, int py) : x(px), y(py) {}
```

Constructor Body:

Simple types are set in the initializer list. Logic that needs multiple steps goes in the body {}.

```
struct DSU {  
    int n;  
    vector<int> par;  
  
    DSU(int n) : n(n), par(n) {  
        // body – runs after initializer list  
        iota(par.begin(), par.end(), 0); // fill 0,1,2,...,n-1  
    }
```

```
};
```

Important Rules:

1. If you define NO constructor at all, C++ gives you a default one but members are garbage on plain declaration.
2. Once you define ANY constructor, C++ no longer gives you the default one for free — you must write it explicitly if needed.

```
struct Point {  
    int x, y;  
    Point(int x, int y) : x(x), y(y) {}  
};
```

```
Point a;           // ERROR – no default constructor  
Point b(3, 4);    // fine
```

3. Methods

A method is a function that belongs to the struct and can directly access its member variables without needing them passed as parameters.

```
struct Point {  
    int x, y;  
  
    Point(int x, int y) : x(x), y(y) {}  
  
    int distSq() {                // directly uses x and y  
        return x*x + y*y;  
    }  
  
    Point translate(int dx, int dy) { // returns a new struct  
        return Point(x+dx, y+dy);  
    }  
};  
  
Point p(3, 4);  
cout << p.distSq();              // 25  
Point q = p.translate(1, 1);    // (4, 5)
```

Without methods you'd need free functions like `distSq(p)` — methods keep related logic inside the struct where it belongs.

4. Operator Overloading

Lets you use operators like +, -, < directly on your struct instead of writing verbose function calls.

```
struct Point {  
    long long x, y;  
  
    Point(long long x, long long y) : x(x), y(y) {}  
  
    Point operator+(Point o) { return {x+o.x, y+o.y}; }  
    Point operator-(Point o) { return {x-o.x, y-o.y}; }  
  
    bool operator<(Point o) const { // const required for sort/set/map
```

```

        if (x != o.x) return x < o.x;
        return y < o.y;
    }

    bool operator==(Point o) const {
        return x == o.x && y == o.y;
    }
};

Point a(1, 2), b(3, 4);
Point c = a + b;    // (4, 6) – uses operator+
Point d = a - b;    // (-2, -2) – uses operator-

vector<Point> v;
sort(v.begin(), v.end()); // works because operator< defined
set<Point> s;              // works because operator< defined

```

The const at the end of operator< and operator== is required whenever the struct is used in set, map, or sort. It tells C++ that this method does not modify the struct.

5. Sorting with Structs

sort() uses operator< internally. Whatever you define in operator< determines the order. The rule: $a < b$ means a comes before b.

```

struct Edge {
    int u, v, w;

    bool operator<(Edge o) const { return w < o.w; }
    // w is MY weight, o.w is the OTHER element's weight
    // w < o.w → I come first if my weight is smaller → ascending order
};

vector<Edge> edges = {{1,2,5}, {2,3,1}, {3,4,3}};
sort(edges.begin(), edges.end());
// result: {2,3,1}, {3,4,3}, {1,2,5} sorted by w ascending

// For descending:
bool operator<(Edge o) const { return w > o.w; }
// now "I come first if my weight is larger" → descending

```

Memory aid: $w < o.w \rightarrow$ ascending, $w > o.w \rightarrow$ descending

6. Common Patterns in CP

Using struct instead of tuple makes code much cleaner:

```

// ugly with tuple
vector<tuple<int,int,int>> edges;
get<2>(edges[0]); // to get weight

// clean with struct
struct Edge { int u, v, w; };
vector<Edge> edges;
edges[0].w; // clean

```

Initializing a vector of structs:

```
struct Edge { int u, v, w; };

vector<Edge> edges;
edges.push_back({1, 2, 5}); // aggregate init – works if no constructor
edges.push_back({2, 3, 1});
```

7. Global vs Local — The Scope Rule

This is the most common mistake. If multiple functions need the struct instance, declare it globally.

```
// WRONG – comb is local to main, solve() cannot see it
int main() {
    Comb comb(200005);
}
void solve() {
    comb.C(10, 3); // ERROR – comb not in scope
}
```

```
// CORRECT – comb is global, all functions can see it
Comb comb(200005); // global

void solve() {
    comb.C(10, 3); // fine
}
int main() {
    // ...
}
```

Rule: if only one function needs it → local is fine. If multiple functions need it → make it global.